

Day 27: File Handling

- File class
- Reading files
- Writing files

1. First understand: What is a File?

Before Java, let's understand what a **file** is.

You already know files such as:

notes.txt

student.txt

data.csv

image.jpg

resume.pdf

A file is basically a place where data is stored **permanently**.

For example:

student.txt

may contain:

Name: Rahul

Age: 21

Course: Java

When the Java program stops, the data inside student.txt still exists.

That's the main difference between:

Variable

String name = "Rahul";

The value exists while the program is running.

File

student.txt

The data remains stored even after the program finishes.

So:

File = permanent storage of data

2. What is File Handling?

File Handling means performing operations on files using a program.

For example:

Create file



Write data



Read data



Modify data



Delete file

Java provides several classes for this.

Some important ones are:

File

FileReader

FileWriter

BufferedReader

BufferedWriter

Scanner

PrintWriter

For today's topic, our main focus is:

File

Reading files

Writing files

3. Why do we need File Handling?

Suppose you create a student management program.

You have:

```
String name = "Rahul";
```

```
int age = 21;
```

If the program closes, these variables disappear.

But suppose you want to store:

Rahul

21

Computer Science

95

for tomorrow.

You need permanent storage.

You can save the information in:

student.txt

Then tomorrow your Java program can read it.

So File Handling allows Java programs to:

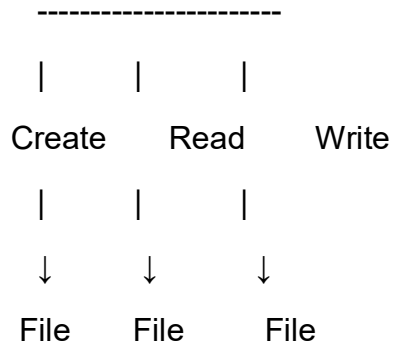
- store information
- retrieve information
- update information
- maintain records
- create reports
- work with configuration files
- process CSV/text files
- store logs

4. File Handling in Java — Big Picture

Think of it like this:

Java Program

|



Java provides classes to communicate with the file system.

5. The File Class

The first important class is:

File

It belongs to:

java.io

So we usually import:

```
import java.io.File;
```

or:

```
import java.io.*;
```

6. What exactly is the File class?

This is VERY important.

When you write:

```
File file = new File("student.txt");
```

You are **not actually creating the file yet**.

You are creating a Java File object that represents the path/name:

student.txt

Think of it as a **reference/address describing a file**.

For example:

```
File file = new File("student.txt");
```

means:

"Java, I want to work with something called student.txt."

It doesn't necessarily mean the physical file has been created.

7. Creating a File

To actually create a file, use:

`createNewFile()`

Example:

```
import java.io.File;
```

```
import java.io.IOException;
```

```
public class Main {
```

```
    public static void main(String[] args) throws IOException {
```

```
        File file = new File("student.txt");
```

```
        if (file.createNewFile()) {
```

```
            System.out.println("File created");
```

```
        } else {
```

```
            System.out.println("File already exists");
```

```
        }
```

```
    }
```

```
}
```

8. Let's understand every line

Line 1

```
import java.io.File;
```

We are importing the File class.

Line 2

```
import java.io.IOException;
```

File operations can cause an input/output problem.

IOException means:

Input/Output Exception

For example:

- file doesn't exist
 - permission problem
 - disk problem
 - reading problem
-

Line 3

```
File file = new File("student.txt");
```

Creates a File object.

Again:

⚠ This does **not necessarily create the physical file**.

Line 4

```
file.createNewFile();
```

Now Java asks the operating system to create:

student.txt

9. Why does createNewFile() return boolean?

This method returns:

true

if a new file was created.

It returns:

false

if the file already existed.

Therefore:

```
if (file.createNewFile()) {  
    System.out.println("File created");  
}
```

means:

Try to create file

↓

Was it newly created?

↓

YES → true

↓

Print "File created"

If already present:

false

and the else executes.

10. Important File Class Methods

You should know these.

exists()

Checks whether the file exists.

```
File file = new File("student.txt");
```

```
System.out.println(file.exists());
```

Output:

true

or:

false

11. getName()

Returns the file name.

```
System.out.println(file.getName());
```

Output:

student.txt

12. getPath()

Returns the path used to create the File object.

```
System.out.println(file.getPath());
```

Possible output:

student.txt

13. getAbsolutePath()

Returns the complete path.

```
System.out.println(file.getAbsolutePath());
```

Example:

C:\Users\User\Documents\Java\student.txt

This is useful when you want to know exactly where Java is looking for the file.

14. length()

Returns the size of the file in bytes.

```
System.out.println(file.length());
```

Example:

120

It means the file occupies approximately 120 bytes.

15. isFile()

Checks whether the path represents a normal file.

```
System.out.println(file.isFile());
```

16. isDirectory()

Checks whether it is a folder/directory.

```
System.out.println(file.isDirectory());
```

For:

student.txt

you would generally get:

false

For a folder:

Documents

you could get:

true

17. delete()

Deletes the file.

```
if (file.delete()) {  
    System.out.println("File deleted");  
}
```

Be careful with this.

Once deleted, the data may not be recoverable normally.

18. Complete File Class Example

```
import java.io.File;
```

```
import java.io.IOException;
```

```
public class Main {
```

```
    public static void main(String[] args) throws IOException {
```

```
        File file = new File("student.txt");
```

```
if (file.createNewFile()) {  
    System.out.println("File created");  
} else {  
    System.out.println("File already exists");  
}  
  
System.out.println("Name: " + file.getName());  
System.out.println("Path: " + file.getPath());  
System.out.println("Absolute Path: " + file.getAbsolutePath());  
System.out.println("Exists: " + file.exists());  
System.out.println("Size: " + file.length() + " bytes");  
System.out.println("Is File: " + file.isFile());  
System.out.println("Is Directory: " + file.isDirectory());  
}  
}
```

19. Relative Path vs Absolute Path

This is important.

Relative path

```
File file = new File("student.txt");
```

You are saying:

Look for student.txt relative to the program's current working directory.

Absolute path

Example:

```
File file = new File("C:\\Users\\User\\Documents\\student.txt");
```

This gives the complete location.

Notice:

C:\Users\User\Documents\student.txt

In Java strings, Windows backslash needs escaping:

"C:\\Users\\User\\Documents\\student.txt"

Why?

Because:

\

has special meaning in Java strings.

20. Now the Most Important Part — Writing to a File

Suppose we have:

student.txt

and want:

Name: Rahul

Age: 21

Course: Java

We need to **write** data into the file.

Java provides:

FileWriter

for this.

Import:

```
import java.io.FileWriter;
```

21. Basic FileWriter Example

```
import java.io.FileWriter;
```

```
import java.io.IOException;
```

```
public class Main {
```

```
    public static void main(String[] args) throws IOException {
```

```
FileWriter writer = new FileWriter("student.txt");

writer.write("Name: Rahul\n");
writer.write("Age: 21\n");
writer.write("Course: Java\n");

writer.close();

System.out.println("Data written successfully");
}
}
```

22. Let's understand it deeply

Step 1

```
FileWriter writer = new FileWriter("student.txt");
```

Creates a FileWriter.

Think:

Java Program

|

FileWriter

|

student.txt

The FileWriter provides a connection between your Java program and the file.

Step 2

```
writer.write("Name: Rahul\n");
```

Writes:

Name: Rahul

into the file.

Step 3

```
writer.close();
```

Closes the connection.

This is important.

23. Why do we need close()?

Imagine opening a water tap.

Open tap



Use water



Close tap

Similarly:

Open file



Write/read



Close file

If you don't close resources properly, you can cause:

- resource leaks
- incomplete writing
- file locking problems
- unnecessary resource usage

So:

```
writer.close();
```

is important.

24. What does \n mean?

```
writer.write("Hello\n");
```

\n means:

Move to the next line.

Therefore:

```
writer.write("Hello\n");
```

```
writer.write("Java\n");
```

creates:

Hello

Java

25. Important FileWriter Behavior — Overwriting

Suppose:

student.txt

already contains:

Hello

Java

Now:

```
FileWriter writer = new FileWriter("student.txt");
```

```
writer.write("New Data");
```

```
writer.close();
```

By default, the existing contents are **overwritten**.

After execution:

New Data

The previous:

Hello

Java

is gone.

26. How to Append Instead of Overwrite?

Sometimes we don't want to erase existing content.

We want:

Old Data

New Data

Use:

```
FileWriter writer = new FileWriter("student.txt", true);
```

The true means:

Append mode.

Example:

```
FileWriter writer = new FileWriter("student.txt", true);
```

```
writer.write("\nNew Student");
```

```
writer.close();
```

Existing:

Student 1

Student 2

becomes:

Student 1

Student 2

New Student

27. Overwrite vs Append

Remember this table:

Code	Behavior
<code>new FileWriter("a.txt")</code>	Overwrite
<code>new FileWriter("a.txt", false)</code>	Overwrite
<code>new FileWriter("a.txt", true)</code>	Append
This is an important interview point.	

28. Now Reading a File

Writing:

Java Program → File

Reading:

File → Java Program

Suppose:

student.txt

contains:

Name: Rahul

Age: 21

Course: Java

We want Java to read this data.

One way is:

FileReader

29. FileReader

Import:

```
import java.io.FileReader;
```

Basic example:

```
import java.io.FileReader;
```

```
import java.io.IOException;
```

```

public class Main {

    public static void main(String[] args) throws IOException {

        FileReader reader = new FileReader("student.txt");

        int data;

        while ((data = reader.read()) != -1) {
            System.out.print((char) data);
        }

        reader.close();
    }
}

```

This looks slightly confusing initially.

Let's break it down carefully.

30. What does reader.read() do?

This is extremely important.

reader.read()

reads **one character at a time**.

Suppose the file contains:

ABC

Internally:

reader.read()

↓

A

reader.read()

↓

B

reader.read()

↓

C

reader.read()

↓

-1

Why -1?

Because:

-1 indicates the end of the file.

31. Why is the return type int?

You may wonder:

If we are reading characters, why does read() return int?

Because Java needs a special value:

-1

to indicate:

End of File

A normal character can be converted from its character code.

So:

```
int data = reader.read();
```

stores the result as an int.

32. Why (char) data?

Suppose:

`data = 65;`

Character code 65 represents:

A

So:

`System.out.print((char) data);`

converts the integer into a character.

Therefore:

`reader.read()`

gets a character code, and:

`(char) data`

converts it to a character.

33. Trace the while loop

Consider:

ABC

Code:

`int data;`

```
while ((data = reader.read()) != -1) {  
    System.out.print((char) data);  
}
```

First iteration

`data = reader.read();`

Suppose:

`data = 65`

65 is:

A

Check:

`65 != -1`

True.

Print:

A

Second iteration

data = 66

Print:

B

Third iteration

data = 67

Print:

C

Fourth iteration

End of file:

data = -1

Check:

-1 != -1

False.

Loop stops.

That's the entire logic.

34. Another Easy Way to Understand Reading

Think:

File

↓

Reader

↓

One character



Check EOF



Print



Next character



...



-1



Stop

35. FileReader is Character-Based

This is important.

FileReader is designed for **character data**.

Good examples:

.txt

.csv

.java

.html

For binary files such as:

.jpg

.png

.mp3

.mp4

.pdf

you generally use byte-oriented streams such as:

FileInputStream

FileOutputStream

We will distinguish these later.

36. BufferedReader

There is another important class:

BufferedReader

It is commonly used for efficient text reading.

Import:

```
import java.io.BufferedReader;
```

Example:

```
import java.io.BufferedReader;
```

```
import java.io.FileReader;
```

```
import java.io.IOException;
```

```
public class Main {
```

```
    public static void main(String[] args) throws IOException {
```

```
        BufferedReader reader =
```

```
            new BufferedReader(new FileReader("student.txt"));
```

```
        String line;
```

```
        while ((line = reader.readLine()) != null) {
```

```
            System.out.println(line);
```

```
        }
```

```
        reader.close();
```

```
}  
}
```

This is very useful.

37. Why BufferedReader?

FileReader reads characters.

BufferedReader provides convenient methods such as:

`readLine()`

Instead of manually reading one character at a time, we can read:

one complete line

38. `readLine()`

Suppose:

Name: Rahul

Age: 21

Course: Java

Then:

```
reader.readLine();
```

returns:

Name: Rahul

Next:

Age: 21

Next:

Course: Java

When there are no more lines:

```
readLine()
```

returns:

null

That's why we write:

```
while ((line = reader.readLine()) != null)
```

39. -1 vs null

This distinction is important.

FileReader

```
reader.read()
```

end of file:

-1

BufferedReader

```
reader.readLine()
```

end of file:

null

Remember:

read() → -1

readLine() → null

40. Why BufferedReader is usually easier

Compare.

FileReader

```
FileReader reader = new FileReader("student.txt");
```

```
int data;
```

```
while ((data = reader.read()) != -1) {  
    System.out.print((char) data);  
}
```

```
reader.close();
```

BufferedReader

```
BufferedReader reader =  
    new BufferedReader(new FileReader("student.txt"));
```

```
String line;
```

```
while ((line = reader.readLine()) != null) {  
    System.out.println(line);  
}
```

```
reader.close();
```

For line-based text files, the second approach is generally much more convenient.

41. BufferedWriter

For writing, we have:

BufferedWriter

Example:

```
import java.io.BufferedWriter;  
import java.io.FileWriter;  
import java.io.IOException;
```

```
public class Main {
```

```
    public static void main(String[] args) throws IOException {
```

```
        BufferedWriter writer =  
            new BufferedWriter(new FileWriter("student.txt"));
```

```
        writer.write("Name: Rahul");
```

```
        writer.newLine();
```

```
writer.write("Age: 21");  
writer.newLine();  
  
writer.write("Course: Java");  
  
writer.close();  
}  
}
```

42. What is newLine()?

Instead of:

```
writer.write("\n");
```

you can use:

```
writer.newLine();
```

Example:

```
writer.write("Hello");
```

```
writer.newLine();
```

```
writer.write("Java");
```

Result:

Hello

Java

43. Why BufferedWriter?

It provides buffering, which can reduce the number of direct operations performed against the underlying file and can improve efficiency, especially with many writes.

Think:

Without buffer:

Java → File

Java → File

Java → File

Java → File

With buffering:

Java

Java

Java

Java

↓

Buffer

↓

File

You don't need to understand the internal implementation yet. Just remember:

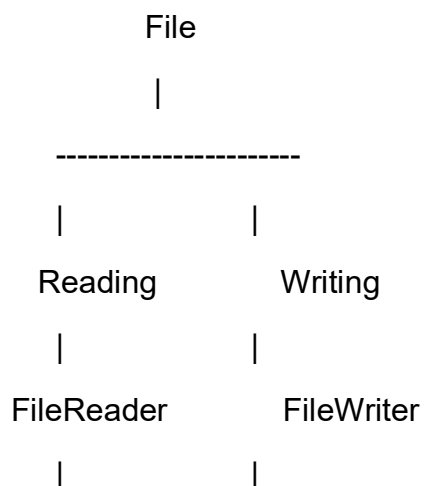
BufferedReader = convenient/efficient text reading

BufferedWriter = convenient/efficient text writing

44. The Connection Between These Classes

This is an important picture:

FILE HANDLING



45. What is java.io?

All these classes come mainly from:

java.io

io means:

Input / Output

Input:

File → Program

Output:

Program → File

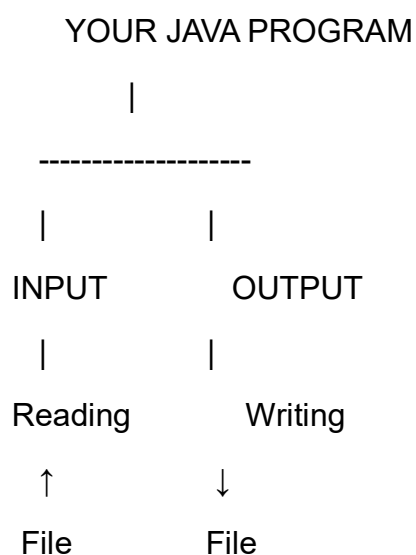
So:

Input = reading

Output = writing

46. Input and Output — Easy Memory Trick

Imagine:



From file to program:

Input

From program to file:

Output

47. Exception Handling in File Handling

File operations can fail.

For example:

```
FileReader reader = new FileReader("abc.txt");
```

What if:

abc.txt

doesn't exist?

Java can throw:

FileNotFoundException

This is why File Handling is strongly connected with Exception Handling.

48. Using throws IOException

You have already seen:

```
public static void main(String[] args) throws IOException
```

This means:

This method may cause an IOException; I'm allowing the exception to be passed to the caller instead of handling it here.

For beginner programs, this is a simple way to make the examples shorter.

49. Using try-catch

Instead of:

```
public static void main(String[] args) throws IOException
```

we can handle it:

```
try {
```

```
    FileReader reader = new FileReader("student.txt");
```

```
    // reading
```

```
        reader.close();
    }
    catch (IOException e) {
        System.out.println("File error occurred");
    }
}
```

This is often preferable when you want to respond to the error.

50. try-with-resources ☆

This is VERY important in real Java code.

Instead of manually doing:

```
reader.close();
```

Java provides:

try-with-resources

Example:

```
try (FileReader reader = new FileReader("student.txt")) {
```

```
    int data;
```

```
    while ((data = reader.read()) != -1) {
```

```
        System.out.print((char) data);
```

```
    }
```

```
}
```

```
catch (IOException e) {
```

```
    System.out.println("Error: " + e.getMessage());
```

```
}
```

Java automatically closes the resource after the try block.

51. Why is it called try-with-resources?

Because we put the resource inside:

```
try ( resource )
```

Example:

```
try (FileReader reader = new FileReader("student.txt")) {
```

Here:

FileReader

is the resource.

When the try block finishes, Java automatically closes it.

52. Recommended Reading Example

For normal text files, this is a very useful pattern:

```
import java.io.BufferedReader;
```

```
import java.io.FileReader;
```

```
import java.io.IOException;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        try (BufferedReader reader =
```

```
            new BufferedReader(new FileReader("student.txt"))) {
```

```
            String line;
```

```
            while ((line = reader.readLine()) != null) {
```

```
                System.out.println(line);
```

```
            }
```

```
    }  
    catch (IOException e) {  
        System.out.println("Error: " + e.getMessage());  
    }  
}  
}
```

53. Recommended Writing Example

```
import java.io.BufferedWriter;  
import java.io.FileWriter;  
import java.io.IOException;  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        try (BufferedWriter writer =  
            new BufferedWriter(new FileWriter("student.txt"))) {  
  
            writer.write("Name: Rahul");  
            writer.newLine();  
  
            writer.write("Age: 21");  
            writer.newLine();  
  
            writer.write("Course: Java");  
  
            System.out.println("Data written successfully");  
        }  
    }  
}
```

```

    }
    catch (IOException e) {
        System.out.println("Error: " + e.getMessage());
    }
}
}

```

54. Complete Example — Create + Write + Read

Let's combine everything.

First:

Create file

↓

Write data

↓

Read data

↓

Display data

Program:

```
import java.io.*;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        File file = new File("student.txt");
```

```
        // Creating file
```

```
        try {
```

```
            if (file.createNewFile()) {
```

```
        System.out.println("File created");
    } else {
        System.out.println("File already exists");
    }
}
catch (IOException e) {
    System.out.println("Unable to create file");
}

// Writing
try (BufferedWriter writer =
    new BufferedWriter(new FileWriter(file))) {

    writer.write("Name: Rahul");
    writer.newLine();

    writer.write("Age: 21");
    writer.newLine();

    writer.write("Course: Java");

}
catch (IOException e) {
    System.out.println("Writing error: " + e.getMessage());
}

// Reading
try (BufferedReader reader =
    new BufferedReader(new FileReader(file))) {
```

```
String line;

while ((line = reader.readLine()) != null) {
    System.out.println(line);
}

}

catch (IOException e) {
    System.out.println("Reading error: " + e.getMessage());
}
}
}
```

55. Understand the Entire Program Flow

When you run it:

Step 1

File file = new File("student.txt");

Java gets a File object representing:

student.txt

Step 2

file.createNewFile();

Creates the actual file if it doesn't exist.

Step 3

new FileWriter(file)

opens the file for writing.

Step 4

writer.write(...)

puts data into the file.

File becomes:

Name: Rahul

Age: 21

Course: Java

Step 5

```
new FileReader(file)
```

opens the file for reading.

Step 6

```
reader.readLine()
```

reads one line at a time.

Step 7

```
System.out.println(line);
```

prints it to the console.

56. Very Important: File Object vs Actual File

Beginners often confuse this.

When you write:

```
File file = new File("student.txt");
```

Think:

File object

↓

represents

↓

"student.txt"

The actual physical file is created when you do:

```
file.createNewFile();
```

or when certain writing APIs create/open the file for writing.

57. Does FileWriter create the file?

Suppose:

student.txt

doesn't exist.

Then:

```
FileWriter writer = new FileWriter("student.txt");
```

can create the file.

So you don't always need:

```
file.createNewFile();
```

before writing.

For example:

```
FileWriter writer = new FileWriter("student.txt");
```

```
writer.write("Hello");
```

```
writer.close();
```

can create student.txt if necessary.

58. File vs FileReader vs FileWriter

Remember this clearly:

File

Used mainly to represent/manage file or directory information.

```
File file = new File("student.txt");
```

It gives methods such as:

```
exists()
```

```
getName()
```

```
length()
```

```
delete()
```

```
isFile()
```

```
isDirectory()
```

FileReader

Used to **read character data**.

```
FileReader reader = new FileReader("student.txt");
```

FileWriter

Used to **write character data**.

```
FileWriter writer = new FileWriter("student.txt");
```

59. FileReader vs BufferedReader

FileReader

Reads characters.

```
int data = reader.read();
```

BufferedReader

Conveniently reads lines.

```
String line = reader.readLine();
```

Think:

FileReader



character by character

BufferedReader



line by line

60. FileWriter vs BufferedWriter

FileWriter

```
writer.write("Hello");
```

BufferedWriter

```
writer.write("Hello");
```

```
writer.newLine();
```

BufferedWriter provides buffering and convenient methods such as:
newLine()

61. What if the file doesn't exist while reading?

Suppose:

student.txt

doesn't exist.

Then:

FileReader reader = new FileReader("student.txt");

can throw:

FileNotFoundException

which is a subclass of:

IOException

Hierarchy:

Exception

|

IOException

|

FileNotFoundException

62. Common File Handling Exceptions

FileNotFoundException

File cannot be found/opened.

IOException

General input/output problem.

For example:

read failure

write failure

file access problem

63. File Handling with Scanner

You can also read a text file using Scanner.

Example:

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        try {
            File file = new File("student.txt");

            Scanner scanner = new Scanner(file);

            while (scanner.hasNextLine()) {
                String line = scanner.nextLine();
                System.out.println(line);
            }

            scanner.close();

        }
        catch (FileNotFoundException e) {
            System.out.println("File not found");
        }
    }
}
```

```
}
```

This is easy for beginners.

64. Scanner vs BufferedReader

Scanner

Easy to understand

Can parse tokens easily

nextInt() available

Can be slower for very large input

Good for simple programs

For today's File Handling fundamentals, learn both, but become comfortable with:
BufferedReader

for line-based file reading.

BufferedReader

Common for efficient text reading

Primarily line-oriented

readLine()

Generally faster for large text input

Common in performance-sensitive reading

65. Writing with PrintWriter

Another useful class:

PrintWriter

Example:

```
import java.io.PrintWriter;
```

```
import java.io.IOException;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        try (PrintWriter writer =
```

```
            new PrintWriter("student.txt")) {
```

```
writer.println("Name: Rahul");  
writer.println("Age: 21");  
writer.println("Course: Java");  
  
}  
catch (IOException e) {  
    System.out.println(e.getMessage());  
}  
}  
}
```

println() automatically adds a new line.

66. Which classes should you learn today?

Don't try to memorize everything at once.

Your core Day 27 knowledge should be:

File

↓

FileReader

↓

BufferedReader

File

↓

FileWriter

↓

BufferedWriter

And also:

try-catch

try-with-resources

IOException

67. Important Methods Cheat Sheet

File

exists()

createNewFile()

getName()

getPath()

getAbsolutePath()

length()

isFile()

isDirectory()

delete()

FileReader

read()

close()

BufferedReader

read()

readLine()

close()

FileWriter

write()

close()

BufferedWriter

write()

newLine()

close()

68. One Very Important Concept — Stream

You will hear the word:

Stream

in Java File Handling.

Don't be scared by the word.

Think of a stream as:

A flow of data between a source and a destination.

For reading:

FILE

↓

DATA STREAM

↓

JAVA PROGRAM

For writing:

JAVA PROGRAM

↓

DATA STREAM

↓

FILE

69. Character Stream vs Byte Stream

This is an important concept for future Java topics.

Character streams

Used mainly for text.

Examples:

FileReader

FileWriter

BufferedReader

BufferedWriter

Byte streams

Used for raw binary data.

Examples:

FileInputStream

FileOutputStream

Think:

Text

↓

Character streams

Images / audio / binary files

↓

Byte streams

70. What is Buffering?

Imagine writing 1000 tiny pieces.

Without buffering:

Write → File

Write → File

Write → File

Write → File

...

With buffering:

Write

Write

Write

Write

↓

Buffer

↓

File

The buffer temporarily holds data and reduces frequent direct interaction with the underlying resource.

You don't need to know the internal implementation for beginner Java.

Just remember:

Buffer = temporary memory used to make I/O operations more efficient/convenient.

71. Common Mistake #1

✗ Wrong:

```
File file = new File("student.txt");
```

and assuming the physical file definitely exists.

Correct understanding:

```
File file = new File("student.txt");
```

creates a File object representing that path.

To explicitly create a new file:

```
file.createNewFile();
```

72. Common Mistake #2

Forgetting:

```
writer.close();
```

Better:

```
try (BufferedWriter writer = ...) {  
    ...  
}
```

because Java automatically closes it.

73. Common Mistake #3

Using:

```
while ((line = reader.readLine()) != null)
```

but forgetting why null is used.

Remember:

```
readLine()
```

↓

```
String
```

↓

```
No more lines?
```

↓

```
null
```

74. Common Mistake #4

Using:

```
while ((data = reader.read()) != -1)
```

and forgetting:

```
(char) data
```

Because:

```
read()
```

returns an integer representing the character or -1 at EOF.

75. Common Mistake #5 — Accidentally Overwriting

This:

```
new FileWriter("student.txt")
```

normally overwrites existing contents.

If you want append:

```
new FileWriter("student.txt", true)
```

76. Common Mistake #6 — Wrong Windows Path

✗

"C:\Users\User\student.txt"

Potentially problematic because backslash is an escape character.

Use:

"C:\\Users\\User\\student.txt"

or use a relative path:

"student.txt"

77. Common Mistake #7 — Reading Before Writing

Suppose you expect:

Name: Rahul

but your program reads:

student.txt

before anything was written.

You may get:

empty file

or file-not-found depending on whether the file exists.

Always understand the order:

Create

↓

Write

↓

Read

when you're building a simple example.

78. Interview Question: What is File Handling?

Answer:

File Handling in Java is the process of creating, reading, writing, updating, and deleting files using Java's I/O classes. It allows programs to store and retrieve data permanently.

79. Interview Question: What is the File class?

Answer:

File is a class from java.io that represents a file or directory path and provides methods to obtain file information and perform operations such as checking existence, creating and deleting files, and retrieving file properties.

80. Interview Question: Does new File() create a file?

Answer:

No. new File() creates a File object representing a file or directory path. The actual file can be explicitly created using createNewFile() or may be created by suitable output APIs such as FileWriter.

81. Interview Question: Difference between FileReader and FileWriter?

FileReader	FileWriter
Reads data	Writes data
Input	Output
Character stream	Character stream
File → Program	Program → File

82. Interview Question: Difference between FileReader and BufferedReader?

FileReader:

Reads characters

BufferedReader:

Provides buffered reading

and convenient line reading using readLine()

Example:

```
BufferedReader reader =  
    new BufferedReader(new FileReader("student.txt"));
```

83. Interview Question: What is IOException?

IOException is an exception related to input/output operations, such as problems while reading from or writing to files.

84. Interview Question: What is try-with-resources?

Try-with-resources is a Java feature that automatically closes resources after the try block finishes, helping prevent resource leaks.

Example:

```
try (BufferedReader reader =  
    new BufferedReader(new FileReader("student.txt"))) {  
  
    // read file  
  
}
```

85. Interview Question: What is append mode?

Append mode means adding new data to the end of existing file contents without replacing the existing contents.

```
FileWriter writer =  
    new FileWriter("student.txt", true);
```

The:

true

enables append mode.

87. The Complete Mental Model

When you see:

File file = new File("student.txt");

think:

"I have a Java object representing this file path."

When you see:

new FileReader(file)

think:

"I want to read character data from this file."

When you see:

new FileWriter(file)

think:

"I want to write character data to this file."

When you see:

BufferedReader

think:

"I want convenient/efficient text reading, especially line by line."

When you see:

BufferedWriter

think:

"I want convenient/efficient text writing."

When you see:

try-with-resources

think:

"Java will automatically close my file resource."

88. One Final Real Example

Let's make a tiny **Student Record File**.

We want the file:

students.txt

to contain:

Student 1

Name: Alice

Age: 20

Course: Java

Student 2

Name: Bob

Age: 21

Course: Java

Program

```
import java.io.*;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        // Writing data
```

```
        try (BufferedWriter writer =
```

```
            new BufferedWriter(new FileWriter("students.txt"))) {
```

```
            writer.write("Student 1");
```

```
            writer.newLine();
```

```
            writer.write("Name: Alice");
```

```
            writer.newLine();
```

```
            writer.write("Age: 20");
```

```
            writer.newLine();
```

```
writer.write("Course: Java");
writer.newLine();

writer.newLine();

writer.write("Student 2");
writer.newLine();

writer.write("Name: Bob");
writer.newLine();

writer.write("Age: 21");
writer.newLine();

writer.write("Course: Java");

}
catch (IOException e) {
    System.out.println("Writing error: " + e.getMessage());
}

// Reading data
try (BufferedReader reader =
    new BufferedReader(new FileReader("students.txt"))) {

    String line;

    while ((line = reader.readLine()) != null) {
```

```
        System.out.println(line);
    }

}

catch (IOException e) {
    System.out.println("Reading error: " + e.getMessage());
}
}
}
```

89. Day 27 — What You Should Know Before Moving On

Make sure you can explain these **without looking at notes**:

Basics

- What is a file?
- What is File Handling?
- Why do we need files?
- What is java.io?
- What is input?
- What is output?
- What is a stream?

File class

- File
- createNewFile()
- exists()
- getName()
- getPath()
- getAbsolutePath()
- length()
- isFile()

- isDirectory()
- delete()

Reading

- FileReader
- BufferedReader
- read()
- readLine()
- -1
- null

Writing

- FileWriter
- BufferedWriter
- write()
- newLine()
- overwrite
- append
- true in FileWriter

Exceptions

- IOException
- FileNotFoundException
- try-catch
- throws
- try-with-resources

90. Day 27 Practice — Do These Yourself 📝💻

Don't immediately look for solutions. Try these in order.

Level 1

1. Create a file

Create:

hello.txt

and print:

File created successfully

Level 2

2. File information

Create:

student.txt

Print:

File name

Absolute path

File size

Whether it exists

Whether it is a file

Level 3

3. Write data

Write:

Name: John

Age: 22

Course: Computer Science

into:

student.txt

Level 4

4. Read data

Read student.txt line by line and display it.

Use:

BufferedReader

Level 5

5. Append data

Initially:

Student 1

Student 2

Then append:

Student 3

without deleting the previous contents.

Level 6

6. Student records

Create:

students.txt

and store 5 students.

Then read the file and display all records.

Level 7 ☆

Create a program that:

1. Create file
2. Write student details
3. Read student details
4. Display file size
5. Display absolute path

Use:

File

BufferedWriter

BufferedReader

try-with-resources

And the simplest rule:

File → file information/operations

FileReader → read characters

BufferedReader → read text line by line

FileWriter → write characters

BufferedWriter → write text efficiently/conveniently

try-with-resources → automatically close resources